



ARCHERIDE_Security_and_RBAC_Model

Security & Role-Based Access Control (RBAC) Model

ARCHERIDE is designed with a **defense-in-depth security posture**, ensuring that access control, data protection, and operational governance are enforced systematically across all layers of the platform. Security is treated as a **foundational requirement for platform availability**, not an optional enhancement.

Authentication Mechanism

ARCHERIDE employs a centralized, credential-based authentication mechanism.

- **Login Methodology:** Users authenticate using their official email address and a securely hashed password.
- **Token-Based Sessions:** Upon successful authentication, the system issues a **JSON Web Token (JWT)** containing the user's identity and role claims.
- **Session Lifecycle:** JWTs are stateless and must be presented in the Authorization header for every protected API request. Tokens are securely stored in client-side session storage to minimize persistence risk.

This approach ensures secure session handling without maintaining server-side session state.

Authorization & RBAC Model

The platform enforces a **least-privilege access model**, with permissions explicitly granted based on defined roles.

Defined Roles

- **Admin**
 - Full platform visibility
 - Global user and system governance authority
 - Oversight of all delivery data and configurations
- **Business Manager (BM)**
 - Acts as the primary **Gatekeeper**
 - Authorizes new user access
 - Oversees project delivery health within assigned regions or functions
 - Authorized to create and update RIDE records
- **Project Manager (PM)**
 - Standard execution role
 - Authorized to create and update RIDE records

Access Restrictions

- Write and delete operations are gated through role checks
- Historical records are transitioned to a **Closed** state rather than deleted, preserving audit integrity

API Security & Middleware Enforcement

- **Endpoint Classification**
 - Public: /auth/login, /api/health
 - Protected: All remaining API routes
- **Authentication Middleware**
 - A centralized Express.js middleware validates JWT signatures and expiration before request execution
- **Unauthorized Access Handling**

- Requests lacking valid authentication are rejected immediately with a **401 Unauthorized** response, preventing business logic execution

Infrastructure Security Controls

- **Encryption:** All client-server communication is protected using **TLS 1.3** with industry-standard AES-256 encryption
- **Reverse Proxy:** Nginx functions as the primary gateway, handling SSL termination and suppressing sensitive server headers
- **Security Headers:** The platform leverages `helmet.js` to enforce CSP, HSTS, X-Frame-Options, and related protections
- **CORS Governance:** API access is restricted to the approved production origin (`ride.arche.global`), blocking unauthorized cross-domain requests

Session Governance

- **Idle Timeout:** Sessions are governed through token expiration
- **Auto-Logout:** Expired or invalid tokens trigger automatic client-side logout and redirection to the login screen, ensuring stale sessions cannot persist

Audit & Accountability

- **Action Attribution:** Every RIDE record is permanently linked to the creating user
- **Traceability:** Creation and modification timestamps enable full historical reconstruction of delivery activity for audit purposes

Security Posture & Validation

- **Security Assessment:** The platform undergoes periodic passive security scans (e.g., OWASP ZAP)
- **Interpretation Statement:** UI-level warnings and framework notices are informational in nature and do not represent exploitable vulnerabilities. Infrastructure-level findings are mitigated through reverse proxy and middleware enforcement.